

# ServiceTitan Integration

## 1. Overview

### 1.1 Description

**ServiceTitan Integration** connects **ServiceTitan** (field service management platform for HVAC, plumbing, and electrical businesses) with **Newo Platform**.

It enables:

- Checking real-time dispatch capacity and presenting available time windows to callers
- Creating bookings that appear on the company's Calls screen for dispatcher action
- Looking up existing customers by phone number for returning callers
- Automatic LLM-powered matching of service requests to business units (HVAC, Plumbing, Electrical)
- Loading and caching business units and job types on session start
- Auto-configuring conversation canvas with service call booking and cancellation scenarios
- OAuth2 client credentials authentication with automatic token refresh on 401

This integration is designed for **home services businesses** (HVAC, plumbing, electrical contractors) who need **an AI receptionist to handle incoming calls, check availability, and create service bookings** through voice or chat.

### 1.2 Use Cases

- When a caller requests a service → Agent identifies the issue, checks dispatch capacity, and creates a booking
- When a caller asks about available times → Agent queries ServiceTitan dispatch capacity filtered by business unit
- When a returning customer calls → Agent looks up their profile by phone number
- When a caller wants to cancel or reschedule → Agent transfers to a human manager
- When a caller asks about job status → Agent collects details and transfers to team member

### 1.3 Supported Features

| Feature                     | Supported | Notes                                                     |
|-----------------------------|-----------|-----------------------------------------------------------|
| Check Dispatch Availability | ✓         | By date range and business unit via Dispatch API          |
| Create Booking              | ✓         | With auto customer creation, appears on Calls screen      |
| Customer Lookup             | ✓         | Search by phone number via CRM API                        |
| LLM Service Classification  | ✓         | Maps service request → business unit via Gemini Gen()     |
| Static Data Sync            | ✓         | Business units + job types loaded on session start        |
| Canvas Auto-Setup           | ✓         | Booking + cancellation + job status scenarios and intents |
| Multi-Business-Unit Support | ✓         | LLM resolves service type → correct business unit         |
| Cancellation                | ✗         | No API for booking cancellation — transfers to manager    |

|                    |   |                                                          |
|--------------------|---|----------------------------------------------------------|
| Reschedule         | ✗ | No direct reschedule — transfers to manager              |
| Job Status Lookup  | ✗ | Intent defined, but transfers to manager (API not wired) |
| Payment Processing | ✗ | Not implemented                                          |

## 2. Requirements

Before installation:

- Active **ServiceTitan** account with API access enabled
- **Client ID** and **Client Secret** from the ServiceTitan Developer Portal (environment-specific)
- **App Key** (ST-App-Key) from the ServiceTitan My Apps portal (environment-agnostic)
- **Tenant ID** from your ServiceTitan account
- At least one **Business Unit** configured in ServiceTitan (e.g., HVAC, Plumbing, Electrical)

## 3. How to Setup

### 3.1 Step 1 — Generate Credentials in ServiceTitan

1. Log in to the **ServiceTitan Developer Portal** (<https://developer.servicetitan.io>)
2. Navigate to **My Apps** → create or select your application
3. Copy the **App Key** (this is environment-agnostic)
4. Go to your application's **Credentials** section
5. Select the environment (Production or Integration/Sandbox)
6. Copy the **Client ID** and **Client Secret**
7. Note your **Tenant ID** from the ServiceTitan account settings

### 3.2 Step 2 — Connect in Platform

1. Open **Newo** → **Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute                      | Required | Default                                                                                             |                |
|--------------------------------|----------|-----------------------------------------------------------------------------------------------------|----------------|
| servicetitan_client_id         | ✓        | —                                                                                                   | O.             |
| servicetitan_client_secret     | ✓        | —                                                                                                   | O.<br>sp       |
| servicetitan_app_key           | ✓        | —                                                                                                   | Ap<br>a        |
| servicetitan_tenant_id         | ✓        | —                                                                                                   | Te             |
| servicetitan_base_url          | ✗        | <a href="https://api.servicetitan.io">https://api.servicetitan.io</a>                               | Al<br>in       |
| servicetitan_auth_url          | ✗        | <a href="https://auth.servicetitan.io/connect/token">https://auth.servicetitan.io/connect/token</a> | O.<br>ht<br>in |
| servicetitan_booking_source    | ✗        | AI Receptionist                                                                                     | Sc             |
| servicetitan_enable_slot_check | ✗        | True                                                                                                | Er             |

|                                        |   |      |          |
|----------------------------------------|---|------|----------|
| servicetitan_enable_booking            | ✗ | True | Er       |
| servicetitan_setup_scenarios           | ✗ | True | At<br>to |
| servicetitan_override_agent_attributes | ✗ | True | O<br>pl  |
| servicetitan_default_business_unit_id  | ✗ | —    | Fa<br>re |

3. Click **Save + Publish All**

4. On publish, the SetupFlow automatically:

- Creates HTTP connectors for ServiceTitan API and token endpoint
- Sets all customer attributes with metadata
- Registers booking and availability tools on ConvoAgent
- Injects availability and booking payload schemas

5. On first conversation, StaticDataFlow loads business units and job types from ServiceTitan

## 4. How to Use

### 4.1 How the Integration Works

The integration uses ServiceTitan **Dispatch API** for availability, **CRM API** for customers and bookings, and **Settings/JPM APIs** for reference data. Service requests are classified to the correct business unit via LLM.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in ServiceTitan

#### Available Triggers

| Trigger                   | Event                            | Description                            |
|---------------------------|----------------------------------|----------------------------------------|
| <b>Check Availability</b> | get_available_slots              | When the agent needs dispatch capacity |
| <b>Create Booking</b>     | book_slot                        | When the agent confirms a booking      |
| <b>Customer Lookup</b>    | search_existing_client           | When identifying a returning caller    |
| <b>Load Static Data</b>   | session_started                  | Auto-load business units and job types |
| <b>Setup</b>              | project_publish_finish           | Initialize integration on deploy       |
| <b>Canvas Setup</b>       | gm_workflow_builder_canvas_built | Auto-add scenarios to canvas           |

#### Available Actions

- **Get Dispatch Capacity** — Check available time windows (GET `/dispatch/v2/tenant/{id}/capacity` )
- **Search Customer** — Find customer by phone (GET `/crm/v2/tenant/{id}/customers?phoneNumber=...` )
- **Create Customer** — New customer record (POST `/crm/v2/tenant/{id}/customers` )
- **Create Booking** — Submit service request to Calls screen (POST `/crm/v2/tenant/{id}/bookings` )
- **Get Business Units** — Load reference data (GET `/settings/v2/tenant/{id}/business-units` )

- **Get Job Types** — Load reference data (GET /jpm/v2/tenant/{id}/job-types )
- **Get Token** — OAuth2 authentication (POST {auth\_url} )

## 4.2 Flows

| Flow                      | Purpose                                                                   |
|---------------------------|---------------------------------------------------------------------------|
| <b>SetupFlow</b>          | Initialize integration: connectors, attributes, schemas, persona          |
| <b>UtilsFlow</b>          | HTTP request execution, Bearer + ST-App-Key headers, token refresh on 401 |
| <b>AvailabilityFlow</b>   | Check dispatch capacity with LLM business unit resolution                 |
| <b>BookingFlow</b>        | Customer search/create → booking creation with contacts                   |
| <b>ExistingClientFlow</b> | Customer lookup by phone number                                           |
| <b>StaticDataFlow</b>     | Load business units + job types on session start                          |
| <b>CanvasSetupFlow</b>    | Auto-add booking, cancellation, job status scenarios to canvas            |
| <b>MetadataFlow</b>       | Version tracking                                                          |

## 4.3 Architecture Diagrams

### Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as ServiceTitan Integration
    participant API as ServiceTitan API

    Note over I,API: Setup (on publish)
    A->>I: project_publish_finish
    I->>I: Create connectors, set attributes, inject schemas

    Note over I,API: Static Data (on session start)
    I->>API: GET /settings/.../business-units
    API-->>I: Business units (HVAC, Plumbing, Electrical)
    I->>API: GET /jpm/.../job-types
    API-->>I: Job types (AC Repair, Drain Cleaning, etc.)
    I->>I: Cache in customer attributes

    Note over U,API: Availability Check
    U->>A: "My AC isn't cooling, need a technician"
    A->>I: get_available_slots {date, time, service_request}
    I->>I: LLM: "AC" → HVAC business unit
    I->>API: GET /dispatch/.../capacity?businessUnitIds=HVAC
    API-->>I: Available time windows
    I->>A: result_success {availability[]}
    A->>U: "We have Monday 9-11 AM or 1-3 PM"

    Note over U,API: Booking
```

```

U->>A: "Monday 9 AM works"
A->>I: book_slot {summary}
I->>API: GET /crm/.../customers?phoneNumber=...
alt Customer not found
    I->>API: POST /crm/.../customers
    API-->>I: customer_id
end
I->>API: POST /crm/.../bookings
API-->>I: booking_id
I->>A: result_success {booking_id}
A->>U: "Your service call is confirmed!"

Note over U,A: Cancellation
U->>A: "I need to cancel my appointment"
A->>U: "I'll connect you with a team member"
A->>A: Transfer to manager

```

## AvailabilityFlow

```

flowchart TD
    E["get_available_slots"] --> GATE1{"Slot check<br>enabled?"}
    GATE1 -->|"No"| SKIP["Return"]
    GATE1 -->|"Yes"| EXTRACT["Extract date, time,<br>service_request"]
    EXTRACT --> LLM1{"service_request<br>provided?"}
    LLM1 -->|"Yes"| RESOLVE["LLM Gen():<br>service -> business_unit_id"]
    LLM1 -->|"No"| DEFAULT["Use default<br>business_unit_id"]
    RESOLVE --> API1
    DEFAULT --> API1
    API1["GET /dispatch/v2/tenant/{id}/capacity<br>startsOnOrAfter=...&businessUnitIds=..."]
    API1 --> OK1{"Success?"}
    OK1 -->|"No"| ERR["result_error"]
    OK1 -->|"Yes"| PARSE["Group slots by date"]
    PARSE --> OUT["result_success<br>{availability[]}"]

```

## BookingFlow

```

flowchart TD
    E["book_slot"] --> GATE1{"Booking<br>enabled?"}
    GATE1 -->|"No"| SKIP["Return"]
    GATE1 -->|"Yes"| PHONE1{"Phone<br>available?"}
    PHONE1 -->|"No"| ASK["urgent_message:<br>ask for phone"]
    PHONE1 -->|"Yes"| SEARCH["GET /crm/v2/tenant/{id}/<br>customers?phoneNumber=..."]
    SEARCH --> FOUND1{"Customer<br>found?"}
    FOUND1 -->|"No"| CREATE["POST /crm/v2/tenant/{id}/<br>customers"]
    CREATE --> CID["customer_id"]
    FOUND1 -->|"Yes"| CID
    CID --> BOOK["POST /crm/v2/tenant/{id}/<br>bookings<br>{source, summary, contacts}"]
    BOOK --> RESULT1{"Success?"}

```

```
RESULT -->|"No"| ERR["result_error"]
RESULT -->|"Yes"| OUT["result_success<br>{booking_id}"]
```

### ExistingClientFlow

```
flowchart TD
    E["search_existing_client"] --> PHONE{{"Phone<br>available?"}}
    PHONE -->|"No"| ERR["result_error:<br>no phone"]
    PHONE -->|"Yes"| API["GET /crm/v2/tenant/{id}/<br>customers?phoneNumber=..."]
    API --> FOUND{{"Customer<br>found?"}}
    FOUND -->|"Yes"| SUCCESS["result_success<br>{customer_id, name, found: true}"]
    FOUND -->|"No"| NOTFOUND["result_success<br>{found: false}"]
```

### StaticDataFlow

```
flowchart TD
    E["session_started"] --> CONF{{"ServiceTitan<br>configured?"}}
    CONF -->|"No"| SKIP["Return"]
    CONF -->|"Yes"| CACHED1{{"Business units<br>already loaded?"}}
    CACHED1 -->|"Yes"| JOB["Skip to job types"]
    CACHED1 -->|"No"| BU["GET /settings/v2/tenant/{id}/<br>business-units"]
    BU --> STORE1["Store servicetitan_business_units"]
    STORE1 --> JOB
    JOB --> CACHED2{{"Job types<br>already loaded?"}}
    CACHED2 -->|"Yes"| DONE["Done"]
    CACHED2 -->|"No"| JT["GET /jpm/v2/tenant/{id}/<br>job-types"]
    JT --> STORE2["Store servicetitan_job_types"]
    STORE2 --> DONE
```

### UtilsFlow — Token Refresh

```
flowchart TD
    REQ["HTTP Request via<br>servicetitan_connector"] --> RESP{{"Status?"}}
    RESP -->|"200"| BODY{{"Body has<br>error/title?"}}
    BODY -->|"No"| SUCCESS["servicetitan_result_success"]
    BODY -->|"Yes"| ERROR["servicetitan_result_error"]
    RESP -->|"401"| REFRESH["POST {auth_url}<br>grant_type=client_credentials"]
    REFRESH --> TOKEN{{"Token<br>received?"}}
    TOKEN -->|"Yes"| SAVE["Save access_token"]
    SAVE --> RETRY{{"Attempt<br>< 3?"}}
    RETRY -->|"Yes"| AGAIN["Retry original request"]
    RETRY -->|"No"| MAXERR["servicetitan_result_error<br>max retries"]
    TOKEN -->|"No"| TOKERR["servicetitan_result_error"]
    RESP -->|"Other"| CONNERR["servicetitan_result_error"]
```

## 4.4 Canvas Scenarios

When `servicetitan_setup_scenarios` is `True` (default), the `CanvasSetupFlow` automatically adds:

### Scenarios:

- **"Scheduling Service Call via Agent"** ( `servicetitan_booking_scenario` ) — 8-step guided booking flow with code-phrases for availability check and booking creation

### Intent Types:

- **"[L] Service Call Request via Agent"** ( `servicetitan_booking_intent` ) — caller wants to schedule service
- **"[T] Job Status Inquiry"** ( `servicetitan_job_status_intent` ) — caller asking about existing job
- **"[T] Cancellation or Reschedule Request"** ( `servicetitan_cancellation_intent` ) — caller wants to cancel/reschedule → transfer to manager

These can be customized on the canvas after setup — modified scenarios are not overwritten.

## 4.5 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice). A mock server is available at `mocks/servicetitan/routes.yaml` for testing without real API credentials.

## 4.6 Testing and Verification

### To test the integration:

1. **Setup:** Publish the project and verify connectors are created
2. **Static Data:** Start a conversation — verify `servicetitan_business_units` and `servicetitan_job_types` are populated
3. **Availability:** Ask about a service — verify dispatch capacity returned with time windows
4. **Booking:** Complete a booking — verify booking created in ServiceTitan Calls screen
5. **Cancellation:** Ask to cancel — verify agent transfers to manager

### If no action occurs:

- Ensure credentials are set ( `servicetitan_client_id` , `servicetitan_client_secret` , `servicetitan_app_key` )
- Verify `servicetitan_tenant_id` is correct
- Confirm `servicetitan_base_url` points to correct environment (production vs sandbox)
- Check feature flags ( `servicetitan_enable_slot_check` , `servicetitan_enable_booking` )
- Re-publish after configuration changes
- Verify `servicetitan_business_units` is populated (check logs for StaticDataFlow)

*Note: This integration creates real bookings in ServiceTitan. Bookings appear on the Calls screen and are visible to dispatchers.*

## 5. FAQ

**Q: What is the difference between a Booking and a Job in ServiceTitan?** A: A **Booking** is an incoming inquiry/request that appears on the Calls screen. A **Job** is a confirmed work order on the dispatch board, created by a dispatcher from a booking. The AI receptionist creates Bookings, not Jobs directly.

**Q: How does the agent determine which business unit to query for availability?** A: The `_resolveBusinessUnitSkill` uses Gemini LLM (Gen, temperature=0.1) to classify the caller's service request against the loaded list of business units and job types. For example, "my AC isn't cooling" → HVAC. If classification fails, it falls back to `servicetitan_default_business_unit_id`.

**Q: Why can't the agent cancel bookings?** A: ServiceTitan's API does not provide an endpoint for cancelling bookings. Cancellation/rescheduling is handled by human dispatchers. The agent recognizes cancellation requests and transfers the caller to a manager.

**Q: How are tokens managed?** A: OAuth2 client credentials flow. Tokens are valid for 15 minutes with no refresh token. On 401 responses, UtilsFlow automatically re-authenticates and retries (up to 3 times). Tokens are cached in `servicetitan_access_token` customer attribute.

**Q: When is static data (business units, job types) loaded?** A: On every `session_started` event (start of each conversation). Data is cached — if already loaded, the API call is skipped. To force reload, clear `servicetitan_business_units` and `servicetitan_job_types` attributes.

**Q: Can I connect to the ServiceTitan Sandbox?** A: Yes. Set `servicetitan_base_url` to `https://api-integration.servicetitan.io` and `servicetitan_auth_url` to `https://auth-integration.servicetitan.io/connect/token`. Use sandbox Client ID and Client Secret.

**Q: What customer information is needed for booking?** A: The caller's phone number is required for customer lookup/creation. Full name is used when creating a new customer. The booking summary is generated from the conversation context describing the service needed.